1. Introduction

Simple graphs play important roles in modeling behaviors of a system. In this paper, we explored properties and enumeration and developed algorithms for counting isomorphism of S-deocrate simple graphs with n vertices. This paper is structured as follows: we begin by exploring a specific choice of n and S, then introduce a relavent result on the number of graphs on n unlabeled vertices. After that, we deduce a theorem for recursively finding the isomorphism class counts for complete graphs and paths of arbitrary n and size of S. Finally, we develop a simple program that enumerates all possible graphs to find isomorphism counts for small n and S.

2. Background

This paper assumes fundamental knowledge of simple graphs and enumeration as in [5] Chapters 1 and 2, and computer science.

Definition 2.1. Let S be a set. An S-deocrated graph is a graph G with a function $l : V(G) \to S$.

Definition 2.2. [5] The adjacency matrix of a loopless graph G with n vertices, written as $A(G)$, is the n-by-n matrix in which entry $a_{i,j}$ is the number of edges in G with endpoints $\{v_i, v_j\}$.

Definition 2.3. [5] An isomorphism from a simple graph G to a simple graph H is a bijection $f : V(G) \to V(H)$ such that $uv \in E(G)$ if and only if $f(u)f(v) \in E(H)$.

Definition 2.4. An isomorphism from S-deocrated graphs (G, l) to (H, l') is an isomorphism f from G to H such that $l(v) = l'(f(v))$ for all $v \in V(G)$.

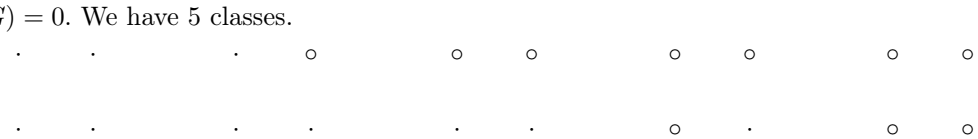
Note that it follows immediately from Definition 2.3 that isomorphic simple graphs are only different by the way vertices are named or positioned. Adjacency relationships between vertices are preserved. Hence, swapping rows and columns in an adjacency matrix gives isomorphic graphs.

We will later introduce an adjusted version of adjacency matrix to better aid our implementation of graph counting algorithm. Also, in an S-decorated simple graph G and function l, for any $v \in V(G)$, we will refer to $l(v)$ as the *attribute* of the vertex.

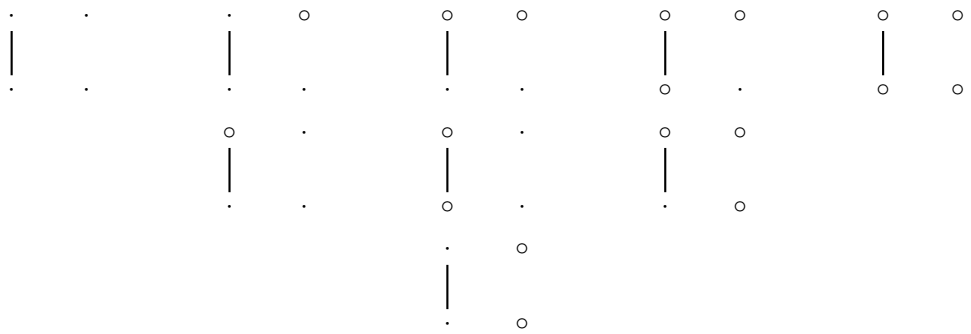
3. A specific case

In this section, we find the number of isomorphism class of simple graph G with 4 vertices and $|S| = 2$. A simple graph with 4 vertices can have upto $\binom{4}{2} = 6$ edges.

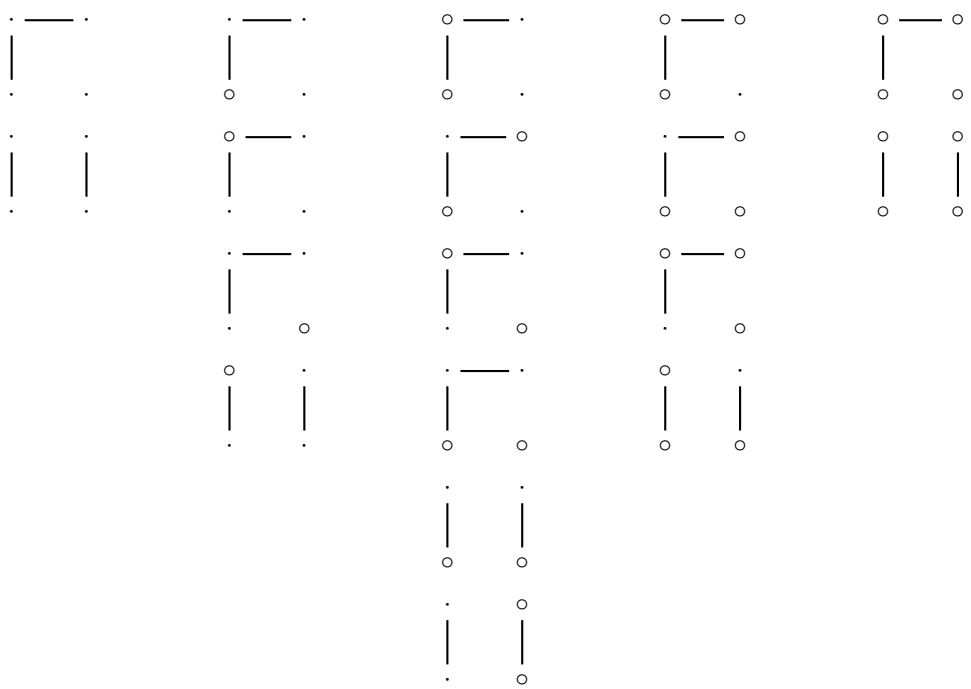
Case 1: $e(G) = 0$. We have 5 classes.



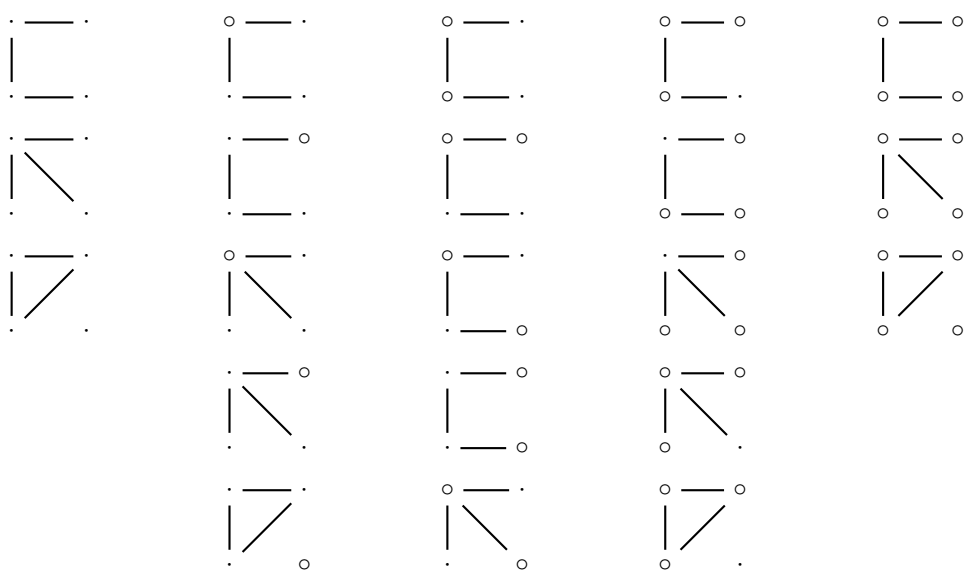
Case 2: $e(G) = 1$. We have 9 classes.

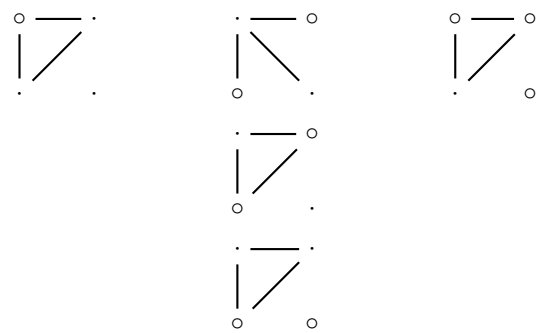


Case 3: $e(G) = 2$. We have 18 classes.



Case 4: $e(G) = 3$. We have 26 classes.





For cases $e(G) = 4$, 5, and 6, they have the same number of isomorphism classes as in Cases $e(G) = 0$, 1 and 2, because one can easily change every edge to non-edge and make every non-edge an edge.

Proposition 3.1. *In an simple S-decorated graph G with function l, n vertices, and i edges, the number of isomorphism classes equals that of an S-deocrated simple graph with the same vertex set (and same function l) and $\binom{n}{2} - i$ edges.*

Proof. Every simple S-decorated graph has i edges and $\binom{n}{2} - i$ non-edges. Switching edges and non-edges preserves vertex attributes and different adjacency relationships between vertices. Hence after switching, non-isomorphic graphs are still non-isomorphic, and each has $\binom{n}{2} - i$ edges.

For every such graph with $\binom{n}{2} - i$ edges, there exists at least one (by injectivity we know it is exactly one) graph on the same vertex set and function l with i edges such that switching edges and non-edges in it results in this graph.

Therefore, we have a bijection between graphs with i edges and those with $\binom{n}{2} - i$ edges. Also notice the fact that switching edges is its own inverse, so we have a bijection. □

Hence, for the 4 vertices and $|S| = 2$ case, we have $2(5 + 9 + 18) + 26 = 90$ isomorphism classes.

4. Relavent results

4.1. **Presenting the results.** [Following ideas in [2], with explanations on fundamental concepts]. In a ring, the integer m such that the sum of m copies of the multiplicative identity equals the additive identity is called the *charactertistic* of the ring. If there is no such m, it has charactertistic zero. [3] Let field $\mathbb{K}$ be charactertistic zero. Let V_n denote the set of undirected graphs with vertices $\{1, ..., n\}$ and edges with weights in $\mathbb{K}$. Consider V_n as a vector space over $\mathbb{K}$ with basis $\mathbf{e}_{1,2}, \mathbf{e}_{1,3}, ..., \mathbf{e}_{n-1,n}$ such that each $\mathbf{e}_{i,j}$ represents an edge between vertices i and j. Note that $\dim(V_n) = \binom{n}{2}$.

Let V_n^* denote the dual space (so $V_n^* = L(V_n, K)$) with dual basis consisting of linear functions $x_{i,j}$ such that $x_{i,j}(\mathbf{e}_{x,y}) = 1$ if $i = x$ and $j = y$ and 0 if $i \neq x$ or $j \neq y$. [1] Let $\mathbb{K}[V_n]$ denote the set of all polynomials that map a vector in V_n to a scalar in $\mathbb{K}$ and let $\mathbb{K}[\{x_{i,j}\}]$ denote the set of all polynomials that map a vector in the dual space to a linear map from V_n to $\mathbb{K}$. We have $\mathbb{K}[V_n] = \mathbb{K}[\{x_{i,j}\}]$ as the article states.

Let S_n be a symmetric group, the group of all permutations of $\{1, ..., n\}$. Let $S_n^{(2)}$ be the pair group of S_n whose actions are induced by S_n. For example, consider $(1\ 2)(3\ 4) \in S_4$. In cycle notation, this permutation swaps 1 and 2, and swaps 3 and 4. Consider the pair $\{1, 3\}$. $(1\ 2)(3\ 4)$ acts on $\{1, 3\}$ and maps it to $\{2, 4\}$. This mapping from $\{1, 3\}$ to $\{2, 4\}$ is an element in $S_4^{(2)}$. Hence, we can say the action of S_n on the set of all $\{i, j\}$ pairs is the same as the action of $S_n^{(2)}$ on the set $\{1, ..., m\}$ such that $m = \binom{n}{2}$, because we have m many different $\{i, j\}$ pairs.

Define the behavior of S_n on V_n and V_n^* as such:

$$\sigma \mathbf{e}_{i,j} = \mathbf{e}_{\sigma(i),\sigma(j)} \quad \sigma^{-1} x_{i,j} = x_{\sigma(i),\sigma(j)} \tag{1}$$

For any $f \in \mathbb{K}[\{x_{i,j}\}]$ of m variables, we say it is an S_n invariant if $\sigma f = f$ for all $\sigma \in S_n$. The S_n invariants form an algebra $\mathbb{K}[V_n]^{S_n}$. The article states that $\mathbb{K}[V_n]^{S_n} \cong \mathbb{K}[\{x_{i,j}\}]^{S_n} \cong \mathbb{K}[x_1, x_2, ..., x_m]^{S_n^{(2)}}$.

Any graph without loops and multiple edges can be expressed as $\sum_{i=1}^{m} g_i \mathbf{e_i}$ where $g_i \in \mathbb{K}$. A simple graph is one with $g_i \in \{0, 1\}$, so denote the set of all simple graphs on n vertices by V_n^0 where $V_n^0 = \{\sum_{i=1}^{m} g_i \mathbf{e_i} | g_i \in \{0, 1\}\} \subset V_n$. Then, we restrict $K[V_n]$ on V_n^0, denoted by $\mathbb{K}[V_n^0]$ such that every polynomial function only take values 0 or 1 on every simple graph. The article indicates $\mathbb{K}[V_n^0]^{S_n^{(2)}}$ is also the restriction of $\mathbb{K}[V_n]^{S_n^{(2)}}$ on V_n^0.

Consider $\mathbb{K}[V_n^0]$ as a vector space. Define $(\mathbb{K}[V_n^0])_i$ as the subspace spanned by elements

$$x_1^{\epsilon_1} x_2^{\epsilon_2} ... x_m^{\epsilon_m} \tag{2}$$

where $\sum_{k=1}^{m} \epsilon_k = i$ and $\epsilon_k \in \{0, 1\}$. Then, it is possible to write $\mathbb{K}[V_n^0]$ as a direct sum:

$$\mathbb{K}[V_n^0] = (\mathbb{K}[V_n^0])_0 + ... + (\mathbb{K}[V_n^0])_m \tag{3}$$

The article further states that $\mathbb{K}[V_n^0]^{S_n^{(2)}} = (\mathbb{K}[V_n^0]^{S_n^{(2)}})_0 + ... + (\mathbb{K}[V_n^0]^{S_n^{(2)}})_m$. The article gives the following conclusion that for each subspace $(\mathbb{K}[V_n^0]^{S_n^{(2)}})_i$, its dimension equals the number of isomorphism classes of simple graphs with n unlabeled vertices and i edges.

4.2. **How this relates to our topic.** This conclusion is highly related to discussing isomorphism classes of S-decorated graphs, because it serves as a base case for when $|S| = 1$. When the size of S increases, the original isomorphism classes would split into new ones, each differed by vertex attributes and new asymmetry created by these. Hence, it is worthwhile to first explore cases for $|S| = 1$ before approaching sets S with more than 1 element. Moreover, this article also provides an approach to invariants underlying isomorphism from the perspectives of abstract and linear algebra. It is possible to study the combinatorial problem using tools from these areas, and existing theorems may apply to accelerate the deduction of conclusions.

5. Exploration on complete graphs

In this section, we explore the number of isomorphism classes on K_n with S of size k.

In a complete graph, every vertex is adjacent to every other vertex; swapping vertex cannot create non-isomorphic graphs. It is the number of vertices of each attribute that creates asymmetry. If $|S| = k$, then we can create an array of length k such that the number at index i indicates the number of vertices having attribute i. The sum of elements in this array is always n.

Theorem 5.1. *Let $F(n, k)$ be the number of isomorphism classes of K_n with $|S| = k$ where $k \geq 1$. Then:*

$$F(n, k) = F(n, k-1) + F(n-1, k) \tag{4}$$

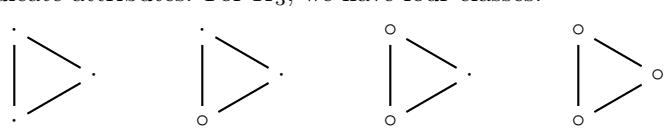
where $F(0, k) = 1$ and $F(n, 1) = 1$.

Proof. Base case: The graph with 0 vertices has only one isomorphism class, because $V(G)$ and $E(G)$ are always empty. For K_n with $|S| = 1$, it is the same as an undecorated K_n graph, and it has only one isomorphism class.

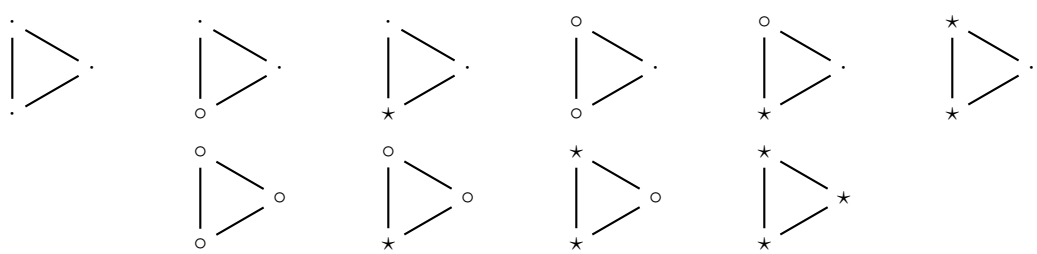
Induction step: In a complete graph, all vertices are adjacent to others. So isomorphism classes are completely determined by the number of appearances of each attribute in S. Let $S = \{s_1, ..., s_k\}$. Suppose there is no vertex with attribute s_1. Then, it is equivalent to find $F(n, k-1)$. Suppose there is at least one vertex with attribute s_1. Then, remove any vertex with this attribute and edges connecting to it. The resulting graph is a smaller complete graph K_{n-1} with k many possible attributes. By inductive hypothesis we know this is $F(n-1, k)$. If there is exactly one vertex, after deletion the smaller graph will have no vertex with attribute s_1, which is caught by the first recursion condition. So we conclude that $F(n, k) = F(n, k-1) + F(n-1, k)$. □

Example 5.2. For K_2 and $k = 2$ we have three classes:

where labels on vertices indicate attributes. For K_3, we have four classes.



When $k = 3$, for K_3 we have ten classes. One can check all examples agree with the theorem.



Below is a Java implementation of the counting method.

```java
/**
* Finds the number of isomorphism classes in an S-decorated complete graph
* @param n the number of vertices
* @param k the size of set S
* @return the number of isomorphism classes
*/
public static int findIsoClassKRecur(int n, int k) {
    if (n == 0 || k == 1) {
        return 1;
    } else {
        return findIsoClassKRecur(n, k - 1)+findIsoClassKRecur(n - 1, k);
    }
}
```

Below is a table of isomorphism class counts for different n's and k's.

$n \setminus k$	1	2	3	4	5	6	7	8	9
0	1	1	1	1	1	1	1	1	1
1	1	2	3	4	5	6	7	8	9
2	1	3	6	10	15	21	28	36	45
3	1	4	10	20	35	56	84	120	165
4	1	5	15	35	70	126	210	330	495
5	1	6	21	56	126	252	462	792	1287
6	1	7	28	84	210	462	924	1716	3003
7	1	8	36	120	330	792	1716	3432	6435
8	1	9	45	165	495	1287	3003	6435	12870
9	1	10	55	220	715	2002	5005	11440	24310

TABLE 1. Number of isomorphism classes in decorated K_n for n's and k's upto 9

6. Exploration on paths

In this section, we explore the number of isomorphism classes on P_n with S of size k.

Given a path P, it is possible to line the vertices up horizontally such that every two vertices are next to each other if and only if they are adjacent. We say P is symmetric if flipping the path results in the same path. Otherwise, it is asymmetric. Also, paths are assumed to be connected.

Example 6.1. The graph on the left below is asymmetric. The right one is symmetric.

$$\cdot - \cdot - \star \qquad \star - \cdot - \star$$

The isomorphism classes are highly related to symmetry in paths, because mirror images are considered to be in the same class and thus we need to cope with duplicates.

Theorem 6.2. *Suppose $|S| = k$. Let s_n denote the number of possible symmetric paths with n vertices, and let a_n denote the number of possible asymmetric ones, upto isomorphism. Then we have:*

$$\begin{aligned} s_n &= s_{n-2}k \\ a_n &= s_{n-2}\binom{k}{2} + a_{n-2}k^2 \end{aligned} \tag{5}$$

where $s_1 = k$ and $a_1 = 0$, and $s_2 = k$ and $a_2 = \binom{k}{2}$.

Proof. Base case: Consider the path with only 1 vertex and k possible attributes. The only vertex is symmetric; flipping it results in the same graph. There are k possible such graphs upto isomorphism, because we cannot map a vertex to one with different attribute. Hence $s_1 = k$ and $a_1 = 0$. For a path with 2 vertices, if two vertices have the same attribute then it is symmetric. There are k possible cases, so $s_2 = k$. Otherwise, it is asymmetric, and there are $\binom{k}{2}$ cases since order doesn't matter. Hence $a_2 = \binom{k}{2}$.

Induction step: Consider a pair of vertices u, v. Given any path P with $n-2$ vertices, we can extend the path by putting each one of u, v at each endpoints. For each of u and v, there are k choices of attribute. The number of u, v pairs with same attribute is k, and that of the pairs with different attributes is $\binom{k}{2}$, because order does not matter. For each u, v pair, the following holds:

Suppose P is symmetric:

Case 1. If $l(v) = l(u)$, then connecting v to the left endpoint and u to the right one is the same as doing otherwise. So we only have 1 possible new path, and this new one is symmetric.

Case 2. If $l(v) \neq l(u)$, then there is only one way to connect endpoints and v and u, and we have 1 possible new path that is asymmetric, upto isomorphism.

Suppose P is asymmmetric.

Case 3. If $l(v) = l(u)$, then we only have one way to connect u and v to two endpoints. Switching them makes no difference. This yields 1 possible longer asymmetric path.

Case 4. If $l(u) \neq l(v)$, we either connect v to the left and u to the right, or connect u to the left and v to the right. The resulting graphs are not mirror images of each other, because the inner vertices are asymmetric. Hence we have 2 possible new paths.

There are s_{n-2} possible symmetric P's, and the only case that creates symmetric graphs is Case 1. Hence $s_n = s_{n-2}k$. Cases 2, 3, and 4 produce asymmetric ones. Case 2 produces $s_{n-2}\binom{k}{2}$ possible cases, Case 3 produces $a_{n-2}k$ paths, and Case 4 generates $a_{n-2} \cdot 2 \cdot \binom{k}{2}$ paths. Adding them up we get $a_n = s_{n-2}\binom{k}{2} + a_{n-2}k + a_{n-2}k(k-1) = s_{n-2}\binom{k}{2} + a_{n-2}k^2$. □

Since s_n and a_n are counted upto isomorphism, the total number of isomorphism classes of a path with n vertices is $s_n + a_n$.

Example 6.3. Consider isomorphism classes of paths with 2 vertices, $|S| = 3$.

$\cdot$ —— $\cdot$ $\quad$ $\cdot$ —— $\circ$ $\quad$ $\cdot$ —— $\star$ $\quad$ $\circ$ —— $\circ$ $\quad$ $\circ$ —— $\star$ $\quad$ $\star$ —— $\star$

There are 3 symmetric ones and 3 asymmetric ones. One can check this agrees with our theorem.

Now consider isomorphism classes of paths with 4 vertices, $|S| = 3$. There are 45 of them, 9 are symmetric and 36 are not. And this agrees with the theorem since $s_4 = 3 \cdot 3 = 9$ and $a_4 = 3 \cdot \frac{3 \cdot 2}{2} + 3 \cdot 3^2 = 36$.

$\cdot - \cdot - \cdot - \cdot$ $\quad$ $\cdot - \cdot - \cdot - \circ$ $\quad$ $\cdot - \cdot - \cdot - \star$ $\quad$ $\cdot - \cdot - \circ - \cdot$ $\quad$ $\cdot - \cdot - \circ - \circ$ $\quad$ $\cdot - \cdot - \circ - \star$

$\cdot - \cdot - \star - \cdot$ $\quad$ $\cdot - \cdot - \star - \circ$ $\quad$ $\cdot - \cdot - \star - \star$ $\quad$ $\cdot - \circ - \cdot - \circ$ $\quad$ $\cdot - \circ - \cdot - \star$ $\quad$ $\cdot - \circ - \circ - \cdot$

$\cdot - \circ - \circ - \circ$ $\quad$ $\cdot - \circ - \circ - \star$ $\quad$ $\cdot - \circ - \star - \cdot$ $\quad$ $\cdot - \circ - \star - \circ$ $\quad$ $\cdot - \circ - \star - \star$ $\quad$ $\cdot - \star - \cdot - \circ$

$\cdot - \star - \cdot - \star$ $\quad$ $\cdot - \star - \circ - \circ$ $\quad$ $\cdot - \star - \circ - \star$ $\quad$ $\cdot - \star - \star - \cdot$ $\quad$ $\cdot - \star - \star - \circ$ $\quad$ $\cdot - \star - \star - \star$

$\circ - \cdot - \cdot - \circ$ $\quad$ $\circ - \cdot - \cdot - \star$ $\quad$ $\circ - \cdot - \circ - \circ$ $\quad$ $\circ - \cdot - \circ - \star$ $\quad$ $\circ - \cdot - \star - \circ$ $\quad$ $\circ - \cdot - \star - \star$

$\circ - \circ - \cdot - \star$ $\quad$ $\circ - \circ - \circ - \circ$ $\quad$ $\circ - \circ - \circ - \star$ $\quad$ $\circ - \circ - \star - \circ$ $\quad$ $\circ - \circ - \star - \star$ $\quad$ $\circ - \star - \cdot - \star$

$\circ - \star - \circ - \star$ $\quad$ $\circ - \star - \star - \circ$ $\quad$ $\circ - \star - \star - \star$ $\quad$ $\star - \cdot - \cdot - \star$ $\quad$ $\star - \cdot - \circ - \star$ $\quad$ $\star - \cdot - \star - \star$

$\star - \circ - \circ - \star$ $\quad$ $\star - \circ - \star - \star$ $\quad$ $\star - \star - \star - \star$

Below is the Java implementation of the theory.

```java
/**
* Finds the number of symmetric and asymmetric possible S-decorated simple paths
* @param n the number of vertices
* @param k the size of set S
* @return an integer array {symmetric count, asymmetric count}
*/
public static int[] findIsoClassPSymRecur(int n, int k) {
    if (n == 1) {
        return new int[]{k, 0};
    } else if (n == 2) {
        return new int[]{k, k*(k-1)/2};
    }
    int[] recurOut = findIsoClassPSymRecur(n-2, k);
    recurOut[1] = recurOut[0]*k*(k-1)/2 + recurOut[1]*k*k;
    recurOut[0] = recurOut[0]*k;
    return recurOut;
}

/**
* Finds the number of isomorphism classes of an S-decorated simple paths
* @param n the number of vertices
* @param k the size of set S
* @return the number of isomorphism classes
*/
public static int findIsoClassPRecur(int n, int k) {
    int[] symasym = findIsoClassPSymRecur(n, k);
    return symasym[0] + symasym[1];
}
```

Below is a table of isomorphism class counts for different n's and k's.

$n \setminus k$	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	1	3	6	10	15	21	28	36	45
3	1	6	18	40	75	126	196	288	405
4	1	10	45	136	325	666	1225	2080	3321
5	1	20	135	544	1625	3996	8575	16640	29889
6	1	36	378	2080	7875	23436	58996	131328	266085
7	1	72	1134	8320	39375	140616	412972	1050624	2394765
8	1	136	3321	32896	195625	840456	2883601	8390656	21526641
9	1	272	9963	131584	978125	5042736	20185207	67125248	193739769

TABLE 2. Number of isomorphism classes in decorated P_n for n's and k's upto 9

7. A simple program

While direct theories and algorithms for counting S-decorated graphs are hard to deduce, it is at least possible to enumerate through all possible combinations using computer programs to find the number of isomorphism classes when n and $|S|$ are small. In this section, we designed such a program to achieve this. The calculation result is attached at the end of this section.

7.1. **Program Design.**

In this section we give a high-level description of the program.

(1) Specify n and k, the number of vertices and the size of S.

(2) Generate all permutations in symmetric group S_n; they are vertex mappings.

(3) Generate all possible vertex attribute assignments as a list of arrays of length n.

(4) Create an empty set L, and let integer isoCount = 0;

(5) For every possible edge combination:

For every vertex attribute assignment:

Create a graph with identity vertex mapping and attribute assignment. If the graph is in L, then discard it. Continue the loop.

If the graph is not in L, then use all possible vertex mapping generated in Step 2 to create a list of graph alias. Add all these to L, and add isoCount by 1. Then conitnue the loop.

(6) Return the value of isoCount.

7.2. **Data Structure.**

7.2.1. *Adjacency matrix and implementation.* An adjacency matrix encodes a graph with a specific vertex labeling. A graph with n vertices can be represented by an n-by-n matrix A such that entry a_{ij} denotes the number of edges from the ith vertex to the jth vertex. In a simple graph, a_{ij} is either 1 or 0, and a_{ii} is always 0. For the sake of implementing the program, we modify this definition. Suppose $S = \{0, 1, 2, ..., k-1\}$ so S has k elements. If there is an edge between the ith vertex and the jth vertex, set a_{ij} to the attribute of the

jth vertex, and set a_{ji} to the attribute of the ith vertex. If there is no edge, then set a_{ij} and a_{ji} to -1. a_{ii} is always equal to the attribute of the ith vertex.

This adjacency matrix is implemented by a byte array of length n^2. A byte is a 4-bit integer with range from -128 to 127, and is the smallest integer type we can use to save memory. Each entry a_{ij} in an adjacency matrix A is given by the $i * n + j$th element in the array. Note that every index now starts from 0, so a_{00} denotes the top left entry in A and the first index in an array of length n^2 is 0 and the last index is $n^2 - 1$. The array implementation is essentially aligning rows in A so they can be written in the same line.

7.2.2. *Vertex map.* A vertex map is a permutation, implemented by a byte array of length n such that the ith element indicates the vertex the ith vertex is mapped to. In a graph object, adjacency matrix is not going to be changed once its created. An identity vertex map is created at the same time, so every vertex is mapped to itself. For any two isomorphic graphs, their adjacency matrix may look different, but their array representing the matrix is the same. To get the entry at ith row and jth column, a method would first convert i and j via the vertex mapping to an array index, then access element in the array at this index. Please note that we do not maintain an actual adjacency matrix. It is the array, the vertex mapping, combined with a getter method that creates the "illusion" of having such a matrix.

Since isomorphic graphs are only different in vertex mapping, it leads to the idea of matrix and matrix alias class, as will be introduced later.

7.2.3. *Vertex attribute assignment.* The assignment is an array of length n such that the ith entry indicates the attribute of the ith vertex.

7.2.4. *Edge code.* An edge code is a binary number that encodes particular edge information. Since a graph can have different vertex labeling and attribute, edge codes are only used for generating adjacency matrices, and it is comparing the matrices that tell us whether two graphs are isomorphic. Edge codes are devised so we have a convenient method to enumerate through all possible edge combinations.

Similar to the idea of using basis vectors to indicate edges in Section 4, we can use a binary number to store the edge information. In a simple graph with n vertices, there are at most $m = \binom{n}{2}$ edges. Hence, we can use m many bits in a long integer, an integer type with 64-bits, to specify edges. For example, the number 101 indicates that in a 3-vertex simple graph, there is an edge between 0th vertex and 1st vertex, no edge between 0th vertex and 2nd vertex, and an edge between 1st vertex and 2nd vertex. The maximum edge code of this 3-vertex is 111. The bijective relation between bit positions and specific edges is irrelavent. The idea is adding 1 continually from 000 to this number gives all possible edge combinations.

7.2.5. *Graph type.* A graph type is a Java enum data that stores the type of a graph.

```
public enum GraphFamily {
    ANY, COMPLETE, PATH
}
```

Graph type specification indicates how a graph object should be initialized and converted to latex code.

7.2.6. *Matrix lookup table.* To identify if a graph is isomorphic to some known graph, we store all known graphs in a lookup table and go through the list to see if they match. The number of graphs to be stored is enourmous. If we use an 1D array of graphs, then the search cost is high. Hence, we implement a hash table for quick lookup. The hash table consists of an array, each entry records the memory address to a separate list. To store a graph, we first calculate its hash code, and use this to find index i, and add the graph to the end of the list stored at the ith position of the array. Hence, we don't have to check graphs stored in other lists.

Below is the implmentation.

```java
import java.util.LinkedList;

public class LinkedListM {
    LinkedList<Graph> data = new LinkedList<>();

    /**
        * Adds a graph to the list
        * @param m the graph
        */
    public void add(Graph m) {
        this.data.addLast(m);
    }

    /**
        * Checks if a graph exists in the list
        * @param m the graph
        * @return true if exists, false otherwise
        */
    public boolean contains(Graph m) {
        for (Graph mat : this.data) {
            if (m.equals(mat)) {
                return true;
            }
        }
        return false;
    }

    public LinkedList<Graph> toLinkedList() {
        return this.data;
    }
}
```

```java
import java.util.ArrayList;
import java.util.List;

public class MatrixLookUpTable {
    LinkedListM[] data;
    int size;
    boolean isCheckingVertexSet;

    /**
        * MatrixLookUpTable constructor
        * @param size the size of the hash table
        * @param isCheckingVertexSet set to true for vertex attributes and edge counts
```

```
        */
    public MatrixLookUpTable(int size, boolean isCheckingVertexSet) {
        this.size = size;
        this.data = new LinkedListM[size];//an array of linkedLists
        this.isCheckingVertexSet = isCheckingVertexSet;
    }

    /**
        * Returns the hash of a graph
        * @param m the graph
        * @return the hash code
        */
    private int determineHash(Graph m) {
        if (this.isCheckingVertexSet) {
            return m.hashCodeVertex();
        } else {
            return m.hashCode();
        }
    }

    /**
        * Adds a graph to the table
        * @param m the graph
        */
    public void add(Graph m) {
        int hash = determineHash(m);
        int index = (hash) % this.size;

        if (this.data[index] == null) {
            this.data[index] = new LinkedListM();
        }
        this.data[index].add(m);
    }

    /**
        * Checks if the graph exists in the table
        * @param m the graph
        * @return true if exists, false otherwise
        */
    public boolean contains(Graph m){
        int hash = determineHash(m);
        int index = (hash) % this.size;
        if (this.data[index] == null) {
            return false;
        }
        return this.data[index].contains(m);
    }

    /**
        * Adds a list of graphs into the table
        * @param ms a list of graphs
        */
    public void addAll(List<Graph> ms) {
        for (Graph m : ms) {
            this.add(m);
        }
    }

    /**
        * Converts all graphs stored to an arrayList
        * @return a list of graphs
        */
```

```java
    public ArrayList<Graph> toArrayList() {
        ArrayList<Graph> result = new ArrayList<>();
        for (LinkedListM lm : this.data) {
            if (lm != null) {
                result.addAll(lm.toLinkedList());
            }
        }
        return result;
    }
}
```

7.2.7. *Hash code.* Hash codes are elementary classification of graphs to determine where they should be stored in a lookup table. Identical graphs (isomorphic graphs with the same vertex labeling) must have the same hash. Also, the image of hash code generating function ideally should cover all spaces of the lookup table, so we can minimize the lookup time by spreading graphs evenly across the place.

There are multiple ways to compute a hash code, and here we use two. Method 1 has a wider image, and method 2 is easier to compute.

Method 1. (Called hashCodeVertex() in graph objects) Multiply every element in the vertex attribute assignment list. Add this product by the return value of Method 2. Add the sum by the number of edges in the graph. Return the number.

Method 2. (Called hashCode() in graph objects) Interpret the first row of adjacency matrix as a base n number. For example, if the graph has 3 vertices, and the first row of the matrix looks like $[2, 3, 1]$, then the return value would be $2 \cdot 3^2 + 3 \cdot 3^1 + 1$.

Please note it is possible that graphs belong to the same isomorphic class might be stored in different places in the lookup table. The main point here is that if two graphs are isomorphic and have identical vertex labeling, they must go to the same list down the table.

7.2.8. *A note on elements of S.* We know if two graphs have different number of vertex attributes, they are not isomorphic. For example, let graphs A, B be simple graphs with 3 vertices, and suppose $S = \{x, y\}$. If A has 2 vertices whose attribute is x but B only has 1, then they must not be isomorphic.

Since the hash code is an integer, we want to encode attribute counts into it, which leads to the idea of using prime numbers and 1. For example, if $|S| = 4$, then $S = \{1, 2, 3, 5\}$. This way, if the attribute counts are different, then hash codes calculated by Method 2 above must be different.

7.3. **The Graph Interface.** In Java, an interface is a protocal for different classes that serve the same role in the entire program. In Graph interface, we specify functions that every class that claims to be a graph should implement.

```java
public interface Graph {
    byte[] getVattributes(); //returns a list of vertex attributes, the set S
    GraphFamily getGraphType(); //returns what the graph type is
    byte getVertexCount(); //returns the number of vertices
    byte getEdgeCount(); //returns the number of edges
```

```
    byte[] getVertexMap(); //returns the vertex mapping
    byte get(int row, int col); //gets a particular entry in adjacency matrix
    int hashCodeVertex(); //gets a hash code that takes S into account
    int hashCode(); //gets a hash code using the first row of adjacency matrix
    void setVertexMapping(byte[] perm); //modifies the vertex mapping
    boolean equals(Graph m); //tests if two adjacency matrices are identical
    String toString(); //converts the adjacency matrix to a string
    String toLatex(); //converts the graph to latex code
}
```

7.4. **Matrix class and matrix alias.** A matrix class is an adjacency matrix with methods specified in Graph interface. A matrix alias class is a class representing the same graph, but with a different vertex labeling. Isomorphic graphs are just graphs with different vertex labeling. A matrix object is correlated to multiple matrix alias objects. A matrix alias does not store data on its own except for its vertex mapping, and it calls methods of its parent matrix object. Matrix alias is designed for balancing the efficiency of graph lookup and memory usage.

Below are implementations for matrix and matrix alias.

```
import java.util.HashMap;

public class Matrix implements Graph {
    private byte[] mat; //1D array implementation of 2D array
    private byte vertexCount;
    private byte[] vertexMap;
    private byte edgeCount;
    private static HashMap<Integer, String> attributeLabel = new HashMap<>();
    static {
        attributeLabel.put(1, "\\cdot");
        attributeLabel.put(2, "\\circ");
        attributeLabel.put(3, "\\star");
        attributeLabel.put(5, "\\diamond");
        attributeLabel.put(7, "\\spadesuit");
        attributeLabel.put(11, "\\triangleleft");
    }
    private int hashVertex = 1;
    private GraphFamily graphType;
    private byte[] vattributes;

    /**
        * Matrix constructor for complete graphs and simple paths
        * @param vertexCount the number of vertices
        * @param vattributes vertex attributes
        * @param type graph type, either complete or path
        * @param vertexMap the initial vertex mapping/permutation
        */
    public Matrix(byte vertexCount, byte[] vattributes,
                            GraphFamily type, byte[] vertexMap) {
        this.graphType = type;
        this.mat = new byte[vertexCount*vertexCount];
        this.vertexCount = vertexCount;
        this.vertexMap = vertexMap;
        this.vattributes = vattributes;

        if (type == GraphFamily.COMPLETE) {
            for (int i = 0; i < vertexCount; ++i) {
                for (int j = 0; j < vertexCount; ++j) {
```

```
                    this.mat[i*vertexCount + j] = vattributes[j];
                }
            }
            this.edgeCount = (byte)(vertexCount * (vertexCount - 1)/2);
        } else if (type == GraphFamily.PATH) {
            for (int i = 0; i < vertexCount; ++i) {
                for (int j = 0; j < vertexCount; ++j) {
                    if (j == i || j == i+1 || j == i-1) {
                        this.mat[i*vertexCount + j] = vattributes[j];
                    } else {
                        this.mat[i*vertexCount + j] = -1;
                    }
                }
            }
            this.edgeCount = (byte)(vertexCount - 1);
        } else {
            throw new IllegalArgumentException("Must specify edge code");
        }
    }

    /**
        * Matrix constructor for any simple graph
        * @param vertexCount the number of vertices
        * @param vattributes vertex attributes
        * @param type graph type, must be any
        * @param vertexMap the initial vertex mapping
        * @param edges the edge code specifying edges
        */
    public Matrix(byte vertexCount, byte[] vattributes,
                    GraphFamily type, byte[] vertexMap, long edges) {
        if (type != GraphFamily.ANY) {
            System.out.println("Graph will be build based on specified edge code");
        }
        this.vertexCount = vertexCount;
        this.vertexMap = vertexMap;
        this.vattributes = vattributes;
        this.graphType = type;
        this.edgeCount = 0;
        int bitPosition = 0;
        this.mat = new byte[vertexCount*vertexCount];
        for (int i = 0; i < vertexCount; ++i) {
            for (int j = i; j < vertexCount; ++j) {
                if (j == i) {
                    this.mat[i*vertexCount + j] = vattributes[j];
                } else {
                    int bitValue = (int) ((edges & 1 << bitPosition) >> bitPosition);
                    if (bitValue == 1) {
                        this.mat[i*vertexCount + j] = vattributes[j];
                        this.mat[j*vertexCount + i] = vattributes[i];
                        this.edgeCount += 1;
                    } else {
                        this.mat[i*vertexCount + j] = -1;
                        this.mat[j*vertexCount + i] = -1;
                    }
                    bitPosition += 1;
                }
            }
        }
    }

    //getter methods
    public byte[] getVattributes() {
```

```
        return this.vattributes;
    }

    public GraphFamily getGraphType() {
        return this.graphType;
    }

    public byte getVertexCount() {
        return this.vertexCount;
    }

    public byte getEdgeCount() {
        return this.edgeCount;
    }

    public byte[] getVertexMap() {
        return this.vertexMap;
    }

    /**
        * Gets a particular entry in adjacency matrix
        * @param row which row
        * @param col which column
        * @return the entry at specified row and column
        */
    public byte get(int row, int col) {
        return this.mat[this.vertexCount*this.vertexMap[row] + this.vertexMap[col]];
    }

    /**
        * Generates hash code with vertex attributes and edge count
        * @return a hash code
        */
    public int hashCodeVertex() {
        this.hashVertex = 1;
        for (int j = 0; j < this.vertexCount; j++) {
            this.hashVertex *= this.vattributes[j];
        }
        return this.hashVertex+this.hashCode()+this.edgeCount;
    }

    /**
        * Generates hash code using the first row of the adjacency matrix
        * @return a hash code
        */
    public int hashCode() {
        int result = 0;
        for (int j = 0; j < this.vertexCount; j++) {
            result *= this.vertexCount;
            result += this.get(0, j);
        }
        return result;
    }

    /**
        * Change the vertex mapping
        * @param perm a vertex mapping
        */
    public void setVertexMapping(byte[] perm) {
        this.vertexMap = perm;
    }

```

```java
    /**
        * Returns the matrix as a String
        * @return the matrix as a String
        */
    public String toString() {
        StringBuilder result = new StringBuilder("[");
        for (int i = 0; i < this.vertexCount; ++i) {
            if (i != 0) {
                result.append(" ");
            }
            result.append("[");
            for (int j = 0; j < this.vertexCount; ++j) {
                result.append(this.get(i, j));
                if (j != this.vertexCount - 1) {
                    result.append(", ");
                } else {
                    result.append("]");
                }
            }
            if (i != this.vertexCount - 1) {
                result.append("\n");
            }
        }
        result.append("]");
        return result.toString();
    }

    /**
        * Compare if two matrices have identical entries
        * @param m a matrix to be compared with
        * @return true if two have identical entries
        */
    public boolean equals(Graph m) {
        if (this.vertexCount != m.getVertexCount() || this.edgeCount != m.getEdgeCount()) {
            return false;
        }
        for (int i = 0; i < this.vertexCount; ++i) {
            for (int j = 0; j < this.vertexCount; ++j) {
                if (this.get(i, j) != m.get(i, j)) {
                    return false;
                }
            }
        }
        return true;
    }

    /**
        * Converts the adjacency matrix to latex code
        * @return the latex code
        */
    public String toLatex() {
        StringBuilder output = new StringBuilder();
        output.append("\\begin{minipage}[b]{");
        output.append(0.14285715);
        output.append("\\linewidth}\n    \\begin{tikzpicture}");
        output.append("[every edge/.style={draw=black, thick},scale=0.7]\n");
        for (int i = 0; i < this.vertexCount; ++i) {
            output.append("        \\node (");
            output.append(i).append(") at (");
            if (this.graphType == GraphFamily.COMPLETE||this.graphType == GraphFamily.ANY) {
                output.append(360/this.vertexCount*i).append(":1)");
            } else if (this.graphType == GraphFamily.PATH) {
```

```
                float separation = 0.8f;
                float leftStart = -((float)this.vertexCount)/2 * separation;
                output.append("0:").append(leftStart + separation*i).append(")");
            }
            int attribute = this.get(i, i);
            output.append(" {$").append(attributeLabel.get(attribute)).append("$};\n");
        }

        for (int i = 0; i < this.vertexCount; ++i) {
            for (int j = 0; j < i; ++j) {
                if (this.get(i, j) != -1) {
                    output.append("        \\path [-] (").append(i);
                    output.append(") edge node[left] {} (");
                    output.append(j).append(");\n");
                }
            }
        }
        output.append("    \\end{tikzpicture}\n\\end{minipage}\n");
        return output.toString();

    }
}
```

```
public class MatrixAlias implements Graph {
    private byte[] vertexMap;
    private Graph original;


    /**
        * Creates an alias of a graph with a different vertex mapping
        * @param graph a graph
        * @param vertexMap a new vertex mapping
        */
    public MatrixAlias(Graph graph, byte[] vertexMap) {
        this.original = graph;
        this.vertexMap = vertexMap;
    }

    //getter methods
    public byte[] getVattributes() {
        return this.original.getVattributes();
    }

    public GraphFamily getGraphType() {
        return this.original.getGraphType();
    }

    public byte getVertexCount() {
        return this.original.getVertexCount();
    }

    public byte getEdgeCount() {
        return this.original.getEdgeCount();
    }

    public byte[] getVertexMap() {
        return this.vertexMap;
    }

    /**
        * Access a particular entry in the adjacency matrix
        * @param row which row
```

```
        * @param col which column
        * @return the entry at the specified row and column
        */
    public byte get(int row, int col) {
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        byte result = this.original.get(row, col);
        this.original.setVertexMapping(originalMap);
        return result;
    }

    /**
        * Generates hash code with vertex attributes and edge count
        * @return a hash code
        */
    public int hashCodeVertex() {
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        int hash = this.original.hashCodeVertex();
        this.original.setVertexMapping(originalMap);
        return hash;
    }

    /**
        * Generates hash code using the first row of the adjacency matrix
        * @return a hash code
        */
    public int hashCode() {
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        int result = this.original.hashCode();
        this.original.setVertexMapping(originalMap);
        return result;
    }

    /**
        * Change the vertex mapping
        * @param perm a vertex mapping
        */
    public void setVertexMapping(byte[] perm) {
        this.vertexMap = perm;
    }


    /**
        * Returns the matrix as a String
        * @return the matrix as a String
        */
    public String toString() {
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        String result = this.original.toString();
        this.original.setVertexMapping(originalMap);
        return result;
    }

    /**
        * Compares if two matrices have identical entries
        * @param m a matrix to be compared with
        * @return true if two have identical entries
        */
    public boolean equals(Graph m) {
```

```
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        boolean result = this.original.equals(m);
        this.original.setVertexMapping(originalMap);
        return result;
    }

    /**
        * Converts the adjacency matrix to latex code
        * @return the latex code
        */
    public String toLatex() {
        byte[] originalMap = this.original.getVertexMap();
        this.original.setVertexMapping(this.vertexMap);
        String latex = this.original.toLatex();
        this.original.setVertexMapping(originalMap);
        return latex;
    }
}
```

7.5. **Main algorithm.** Here is the implementation of the program, together with implementations of the theories aforementioned.

```
import java.util.ArrayList;

public class Main {
    public static byte[] primeNumbers = {1, 2, 3, 5, 7, 11, 13, 17, 19,
            23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97};

    /**
        * Removes a certain element from an array
        * @param nums the array
        * @param n the element to be removed
        * @return a shorter array in the same order with n deleted
        */
    public static byte[] removeEle(byte[] nums, byte n) {
        int index = -1;
        for (int i = 0; i < nums.length; ++i) {
            if (nums[i] == n) {
                index = i;
                break;
            }
        }

        byte[] result = new byte[nums.length-1];
        int newIndex = 0;
        for (int i = 0; i < nums.length; i++) {
            if (i != index) {
                result[newIndex] = nums[i];
                newIndex++;
            }
        }
        return result;
    }

    /**
        * Converts a list of graphs to latex code
        * @param lst a list of graphs
        */
    public static void printAllLatex(ArrayList<Graph> lst) {
```

```java
        StringBuilder latex = new StringBuilder();
        latex.append("\\begin{center}\n");
        for (int i = 0; i < lst.size(); ++i) {
            latex.append(lst.get(i).toLatex());
            if (i % 6 == 5) {
                latex.append("\\end{center}\n\\begin{center}\n");
            }
        }
        latex.append("\\end{center}\n");
        System.out.println(latex);
    }

    /**
        * Generates a list of all possible permutations
        * @param nums a list of unique integers
        * @return a list of all possible permutations
        */
    public static ArrayList<byte[]> permutation(byte[] nums) {
        ArrayList<byte[]> output = new ArrayList<>();
        if (nums.length == 0) {
            output.add(new byte[]{});
            return output;
        }

        for (byte n : nums) {
            //remove the selected element, n
            byte[] temp = nums.clone();
            temp = removeEle(temp, n);
            ArrayList<byte[]> reOut = permutation(temp);
            for (byte[] restP : reOut) {
                //add n back
                byte[] addedFirst = new byte[restP.length+1];
                addedFirst[0] = n;
                System.arraycopy(restP, 0, addedFirst, 1, restP.length);
                output.add(addedFirst);
            }
        }
        return output;
    }

    /**
        * Generates an array of ascending integers from 0
        * @param n the number of elements
        * @return an array starting from 0 to n-1
        */
    public static byte[] generateAscending(int n) {
        byte[] ascending = new byte[n];
        for (byte i = 0; i < n; ++i) {
            ascending[i] = i;
        }
        return ascending;
    }

    /**
        * Creates all possible graphs with different vertex labeling
        * @param mat the graph
        * @param perms all possible vertex permutations
        * @return a list of all possible isomorphic graphs
        */
    public static ArrayList<Graph> allMatrixSwaps(Graph mat, ArrayList<byte[]> perms) {
        MatrixLookUpTable result = new MatrixLookUpTable(perms.size()*3, false);
        Graph temp;
```

```
        for (byte[] permMap : perms) {
            temp = new MatrixAlias(mat, permMap);
            temp.setVertexMapping(permMap);
            //if not appeared before, add to result
            if (!result.contains(temp)) {
                result.add(temp);
            }
        }
        return result.toArrayList();
    }

    /**
        * Generates a list of all possible vertex attributes
        * @param vcount the number of vertices
        * @param scount the size of set S
        * @return list of all possible arrays; ith vertex has attribute at ith index
        */
    public static ArrayList<byte[]> varVertexAttribute(byte vcount, byte scount) {
        if (vcount == 0) {
            ArrayList<byte[]> output = new ArrayList<>();
            output.add(new byte[]{});
            return output;
        }
        ArrayList<byte[]> output = new ArrayList<>();
        ArrayList<byte[]> reOut = varVertexAttribute((byte) (vcount -1), scount);
        for (byte i = 0; i < scount; ++i) {
            for (byte[] a : reOut) {
                byte[] addedFirst = new byte[a.length+1];
                addedFirst[0] = primeNumbers[i];
                System.arraycopy(a, 0, addedFirst, 1, a.length);
                output.add(addedFirst);
            }

        }
        return output;
    }

    /**
        * Finds the number of S-decorated simple paths upto isomorphism through enumeration
        * @param vcount the number of vertices
        * @param scount the size of set S
        * @param needLatex set to true to print latex code for graphs found
        * @return the number of paths upto isomorphism
        */
    public static int findIsoClassPEnum(byte vcount, byte scount, boolean needLatex) {
        ArrayList<byte[]> vertexAttribute = varVertexAttribute(vcount, scount);
        MatrixLookUpTable possibleGraphs =
            new MatrixLookUpTable((int)(Math.pow(scount*vcount, 3)), true);
        ArrayList<Graph> isoClasses = new ArrayList<>();
        int isoCount = 0;
        byte[] vertexList = generateAscending(vcount);
        ArrayList<byte[]> allperms = permutation(vertexList);

        int totalAttriVar = vertexAttribute.size();
        int progress = 0;
        for (byte[] attributeset : vertexAttribute) {
            Graph graph =
                new Matrix(vcount, attributeset, GraphFamily.PATH, allperms.getFirst());
            if (!possibleGraphs.contains(graph)) {
                isoCount += 1;
                isoClasses.add(graph);
                possibleGraphs.addAll(allMatrixSwaps(graph, allperms));
```

```
            }
            //progress bar
            progress += 1;
            if (progress % 100 == 0) {
                System.out.print("Tried "+progress+" out of "+totalAttriVar);
                System.out.print(" ("+(float)progress/totalAttriVar*100+"%)");
                System.out.print("\r");
            }
        }

        if (needLatex) {
            printAllLatex(isoClasses);
        }
        return isoCount;
    }

    /**
        * Finds the number of decorated complete graphs upto isomorphism through enumeration
        * @param vcount the number of vertices
        * @param scount the size of set S
        * @param needLatex set to true to print latex code of graphs found
        * @return the number of complete graphs upto isomorphism
        */
    public static int findIsoClassKEnum(byte vcount, byte scount, boolean needLatex) {
        ArrayList<byte[]> vertexAttribute = varVertexAttribute(vcount, scount);
        MatrixLookUpTable possibleGraphs =
                                new MatrixLookUpTable((int)(Math.pow(scount*vcount, 3)), true);
        ArrayList<Graph> isoClasses = new ArrayList<>();
        int isoCount = 0;
        byte[] vertexList = generateAscending(vcount);
        ArrayList<byte[]> allperms = permutation(vertexList);

        int totalAttriVar = vertexAttribute.size();
        int progress = 0;
        for (byte[] attributeset : vertexAttribute) {
            Graph graph =
                new Matrix(vcount, attributeset, GraphFamily.COMPLETE, allperms.getFirst());
            if (!possibleGraphs.contains(graph)) {
                isoCount += 1;
                isoClasses.add(graph);
                possibleGraphs.addAll(allMatrixSwaps(graph, allperms));
            }
            progress += 1;
            if (progress % 100 == 0) {
                System.out.print("Tried " + progress + " out of " + totalAttriVar);
                System.out.print(" (" + (float)progress/totalAttriVar*100 + "%)" + "\r");
            }
        }

        if (needLatex) {
            printAllLatex(isoClasses);
        }
        return isoCount;
    }


    /**
        * Generates the maximum edge code
        * @param vcount the number of vertices
        * @return the maximum edge code
        */
    public static long generateEdgeCodeUpperBound(byte vcount) {
```

```java
        if (vcount == 1) {
            return 0L;
        }
        int edgeNumber = vcount * (vcount-1) /2;
        return generateEdgeCodeUpperBoundHelper((byte) (edgeNumber-1));
    }

    /**
        * Generates the maximum edge code
        * @param n the maximum number of edges
        * @return the maximum edge code
        */
    public static long generateEdgeCodeUpperBoundHelper(byte n) {
        if (n == 0) {
            return 1L;
        } else {
            return generateEdgeCodeUpperBoundHelper((byte) (n-1)) * 2 + 1;
        }
    }

    /**
        * Finds the number of S-decorated simple graphs upto isomorphism
        * @param vcount the number of vertices
        * @param scount the size of set S
        * @param needLatex set to true to print latex code of graphs found
        * @return the number of graphs upto isomorphism
        */
    public static long findIsoClassGeneralEnum(byte vcount, byte scount, boolean needLatex) {
        ArrayList<byte[]> vertexAttribute = varVertexAttribute(vcount, scount);
        MatrixLookUpTable possibleGraphs =
            new MatrixLookUpTable((int)(Math.pow(scount*vcount, 4)+7), true);
        ArrayList<Graph> isoClasses = new ArrayList<>();
        long isoCount = 0;
        byte[] vertexList = generateAscending(vcount);
        ArrayList<byte[]> allperms = permutation(vertexList);
        int totalAttriVar = vertexAttribute.size()*(int)Math.pow(2, vcount * (vcount-1)/2);
        int progress = 0;

        long startTime = System.currentTimeMillis();

        for (long edgeCode = 0; edgeCode <= generateEdgeCodeUpperBound(vcount); edgeCode++) {
            for (byte[] attributeset : vertexAttribute) {
                Graph graph =
                    new Matrix(vcount, attributeset,
                        GraphFamily.ANY, allperms.getFirst(), edgeCode);
                if (!possibleGraphs.contains(graph)) {
                    isoCount += 1;
                    isoClasses.add(graph);
                    possibleGraphs.addAll(allMatrixSwaps(graph, allperms));
                }
                //progress bar
                progress += 1;
                if (progress % 500 == 0) {
                    long nowTime = System.currentTimeMillis();
                    float eta =
                            (totalAttriVar-progress)
                                /(float)progress * (nowTime - startTime)/1000;

                    System.out.print("Tried " + progress + " out of " + totalAttriVar);
                    System.out.print(" (" + (int)(((float)progress/totalAttriVar)*100));
                    System.out.print("%) Count: " + isoCount);
                    System.out.print(" ETA: " + (int)eta/3600 + ":" + (int)(eta/60) % 60);
```

```
                    System.out.print(":" + eta%60 + " \r");
                }
            }
        }

        if (needLatex) {
            printAllLatex(isoClasses);
        }
        return isoCount;
    }

    /**
        * Using theorem, finds the number of S-decorated complete graph upto isomorphism
        * @param n the number of vertices
        * @param k the size of set S
        * @return the number of isomorphism classes
        */
    public static int findIsoClassKRecur(int n, int k) {
        if (n == 0 || k == 1) {
            return 1;
        } else {
            return findIsoClassKRecur(n, k -1)+findIsoClassKRecur(n-1, k);
        }
    }

    /**
        * Using theorem, finds number of symmetric and asymmetric S-decorated simple paths
        * @param n the number of vertices
        * @param k the size of set S
        * @return an integer array {symmetric count, asymmetric count}
        */
    public static int[] findIsoClassPSymRecur(int n, int k) {
        if (n == 1) {
            return new int[]{k, 0};
        } else if (n == 2) {
            return new int[]{k, k*(k-1)/2};
        }
        int[] recurOut = findIsoClassPSymRecur(n-2, k);
        recurOut[1] = recurOut[0]*k*(k-1)/2 + recurOut[1]*k*k;
        recurOut[0] = recurOut[0]*k;
        return recurOut;
    }

    /**
        * Using theorem, finds number of isomorphism classes of S-decorated simple paths
        * @param n the number of vertices
        * @param k the size of set S
        * @return the number of isomorphism classes
        */
    public static int findIsoClassPRecur(int n, int k) {
        int[] symasym = findIsoClassPSymRecur(n, k);
        return symasym[0] + symasym[1];
    }


    public static void main(String[] args) {
        byte n = 4;
        byte s = 2;
        //example usage
        System.out.println("For " + n + " vertices and |S| = "+s +
                "we have "+findIsoClassGeneralEnum(n, s, true)+" isomorphisms");

```

```
    }
}
```

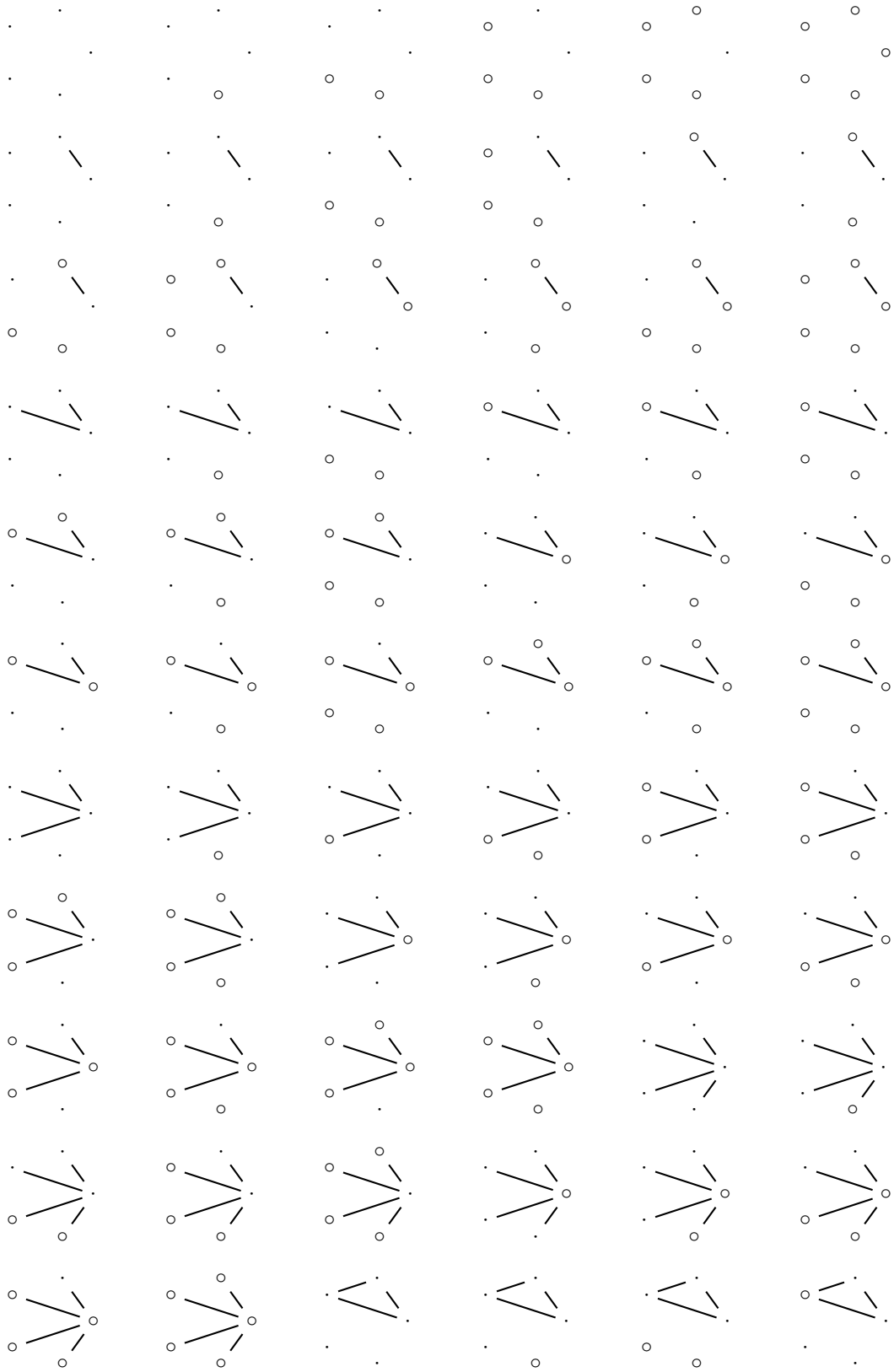
7.6. **Calculation results.** Below is a table of isomorphism class numbers calculated so far. The calculation cost grows rapidly, so it is only possible to run the program on small graphs.

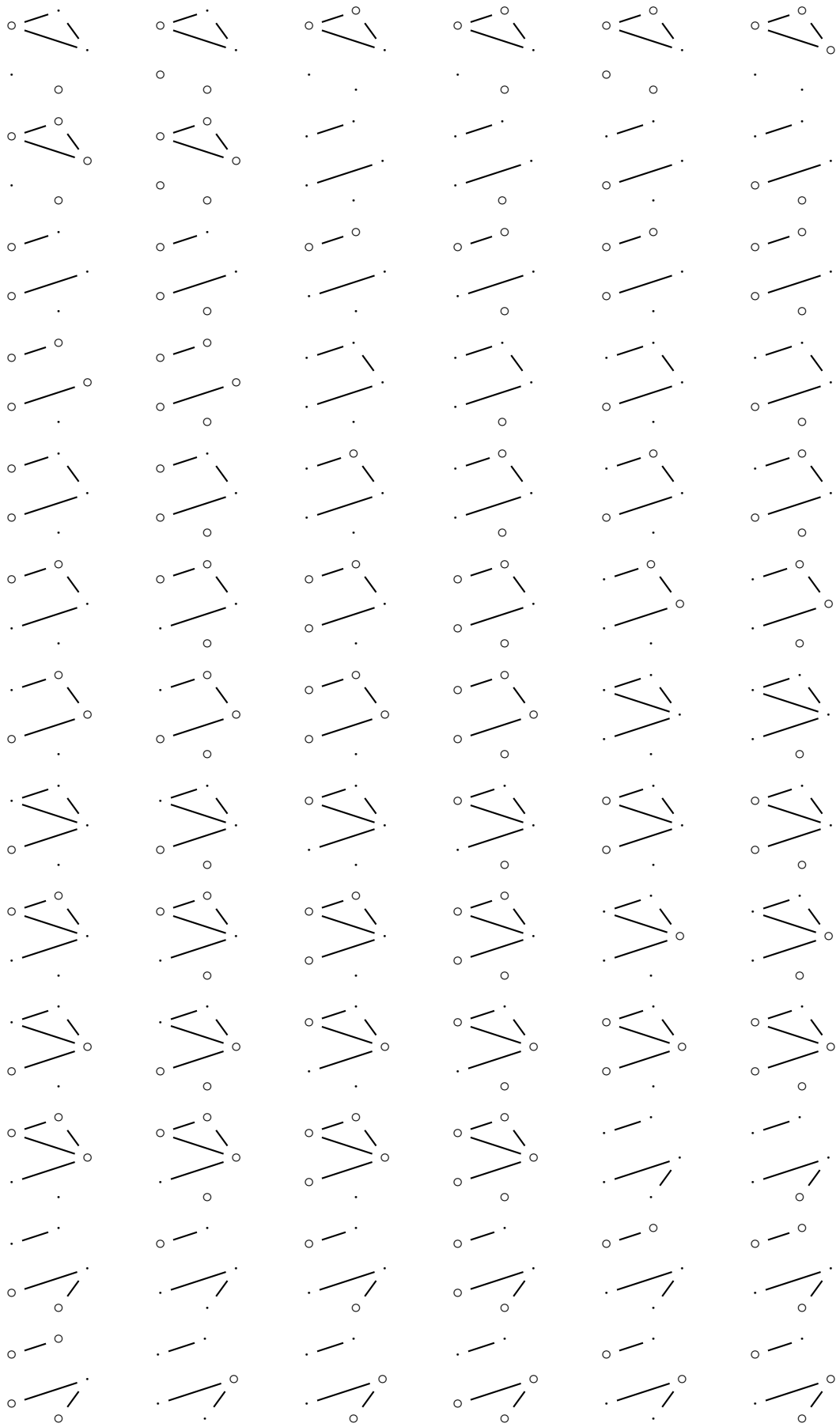
$n \setminus k$	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	6	12	20	30	42	56	72	90
3	4	20	56	120	220	364	560	816	1140
4	11	90	357	996	2255	4446	7945	13192	20691
5	34	544	3258	12208	34754	82608	172858	328992	581922
6	156	5096	47324	241520					
7	1044								

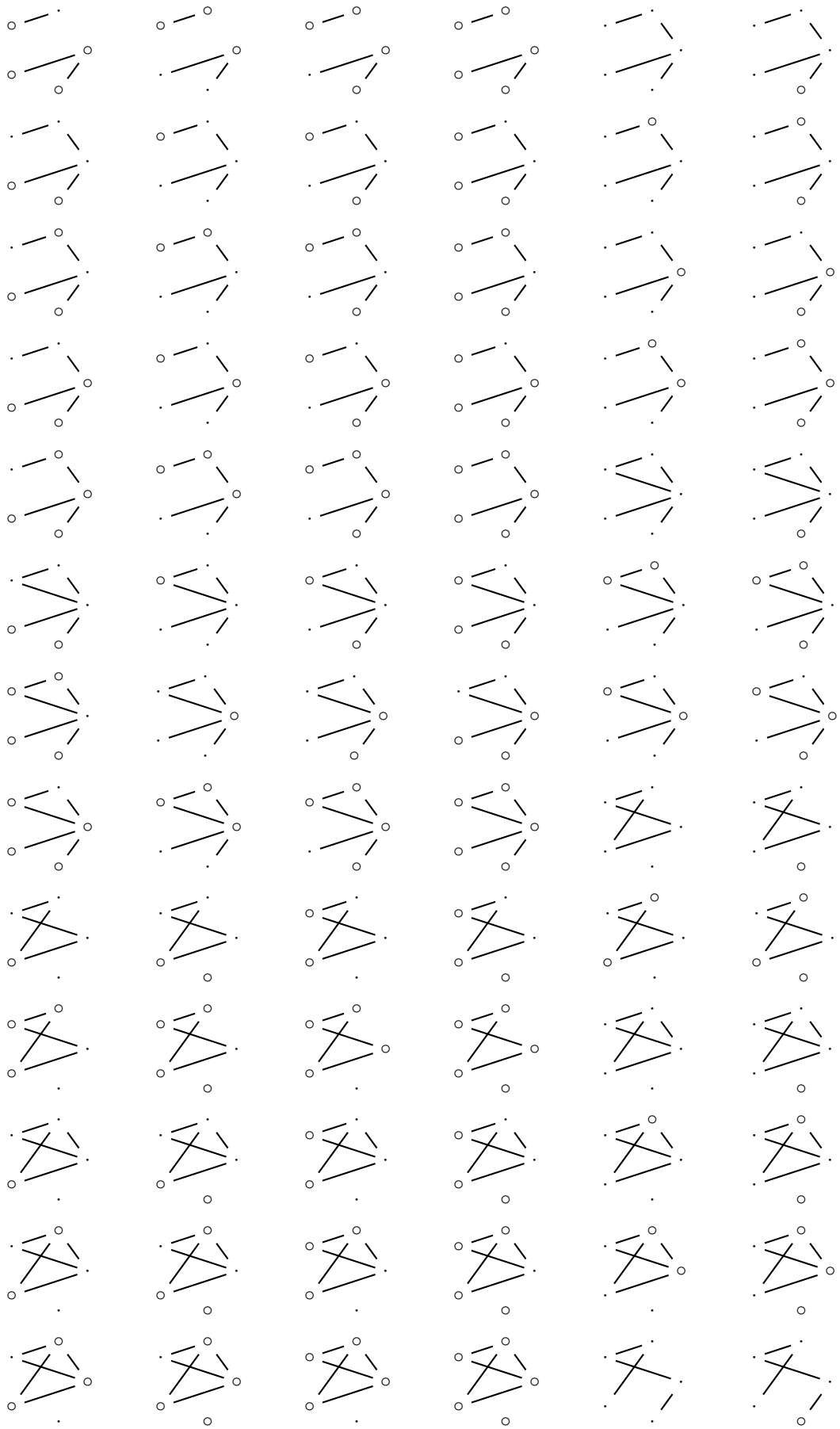
Note for $k = 1$ cases, the table agrees with the A000088 table about graphs on n unlabeled vertices [4].

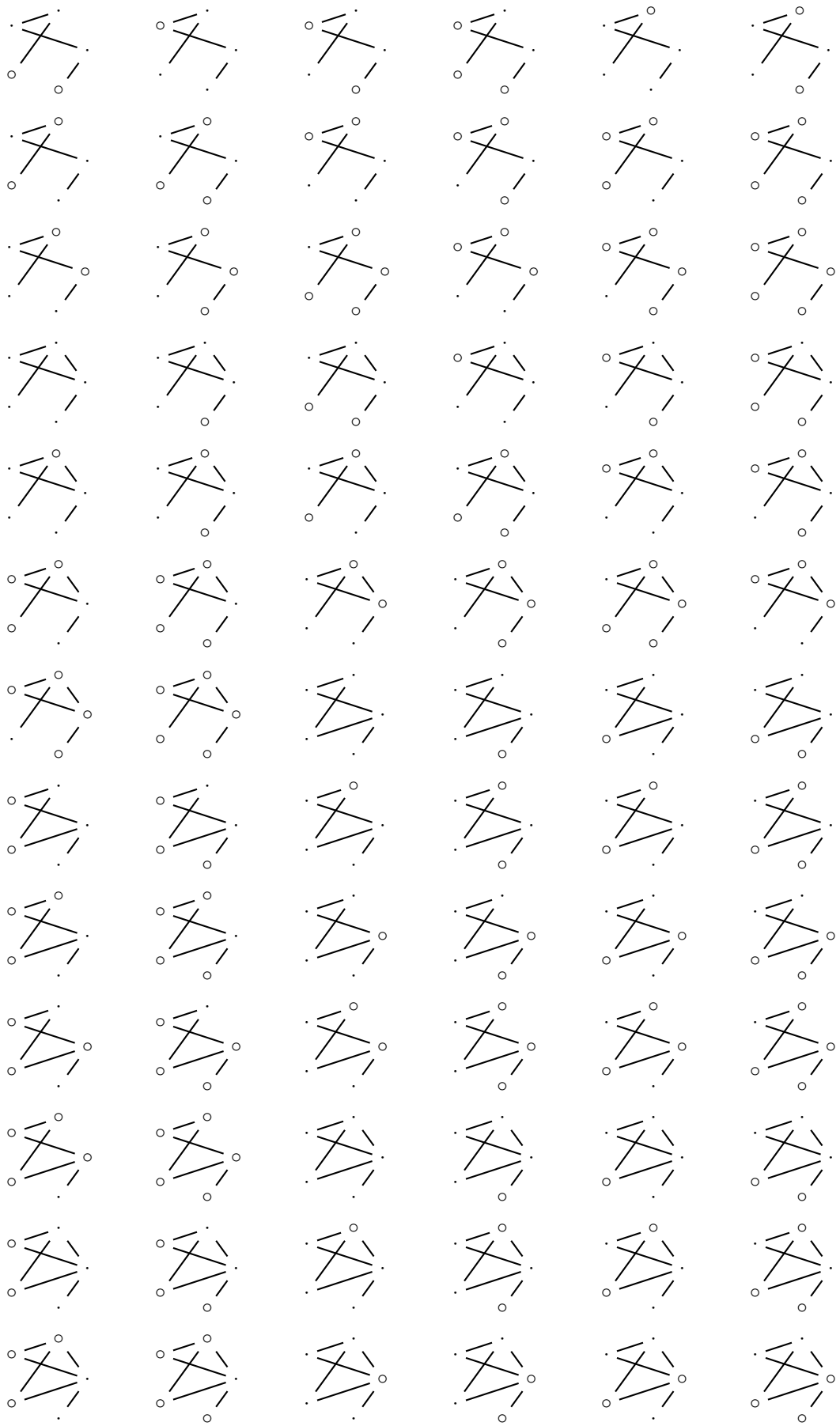
8. Appendix: All simple graphs for $n = 5$ and $|S| = 2$

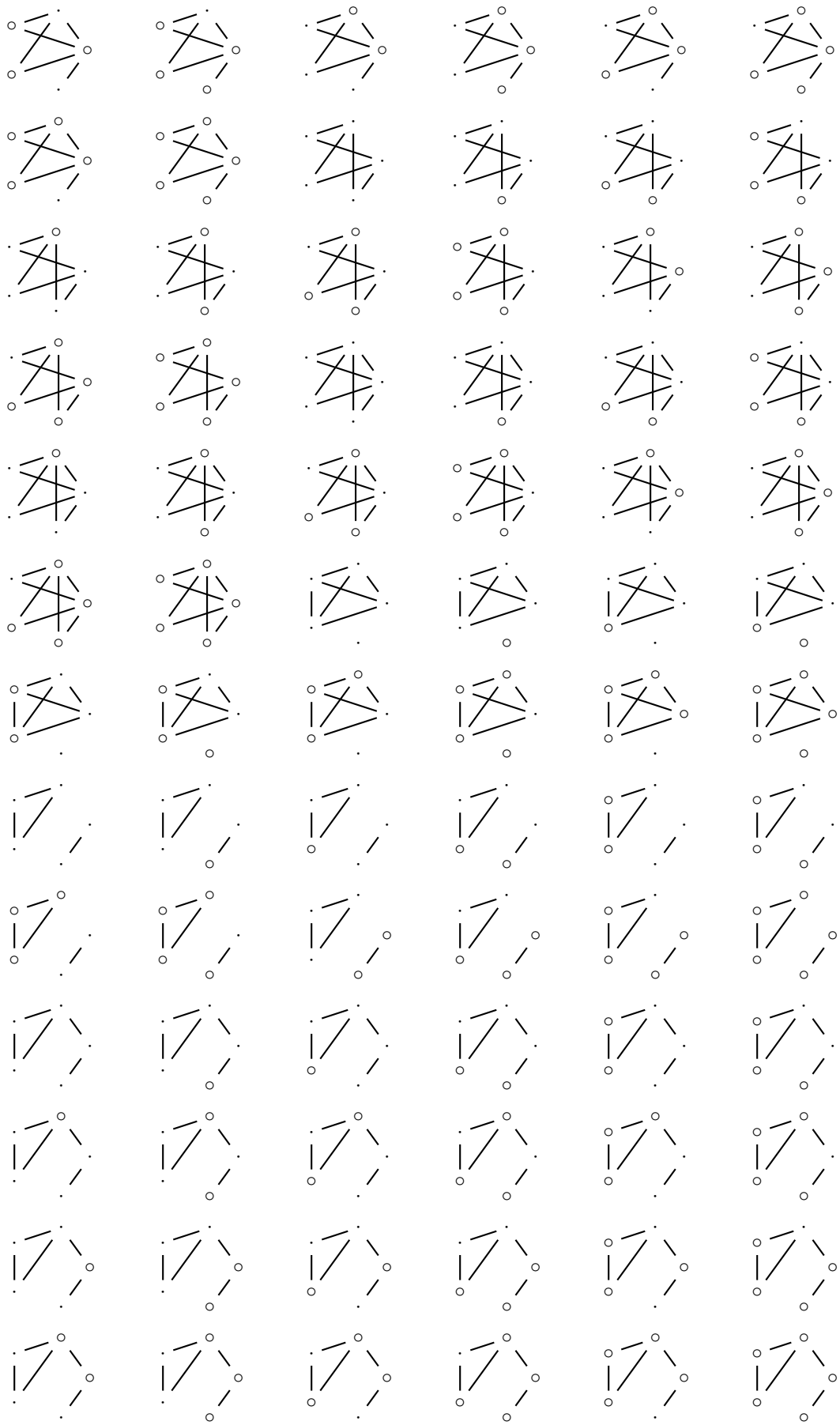
Out of deep appreciation for the beauty of graphs, I hereby print all 544 graphs generated by the program for the case $n = 5$ and $|S| = 2$.

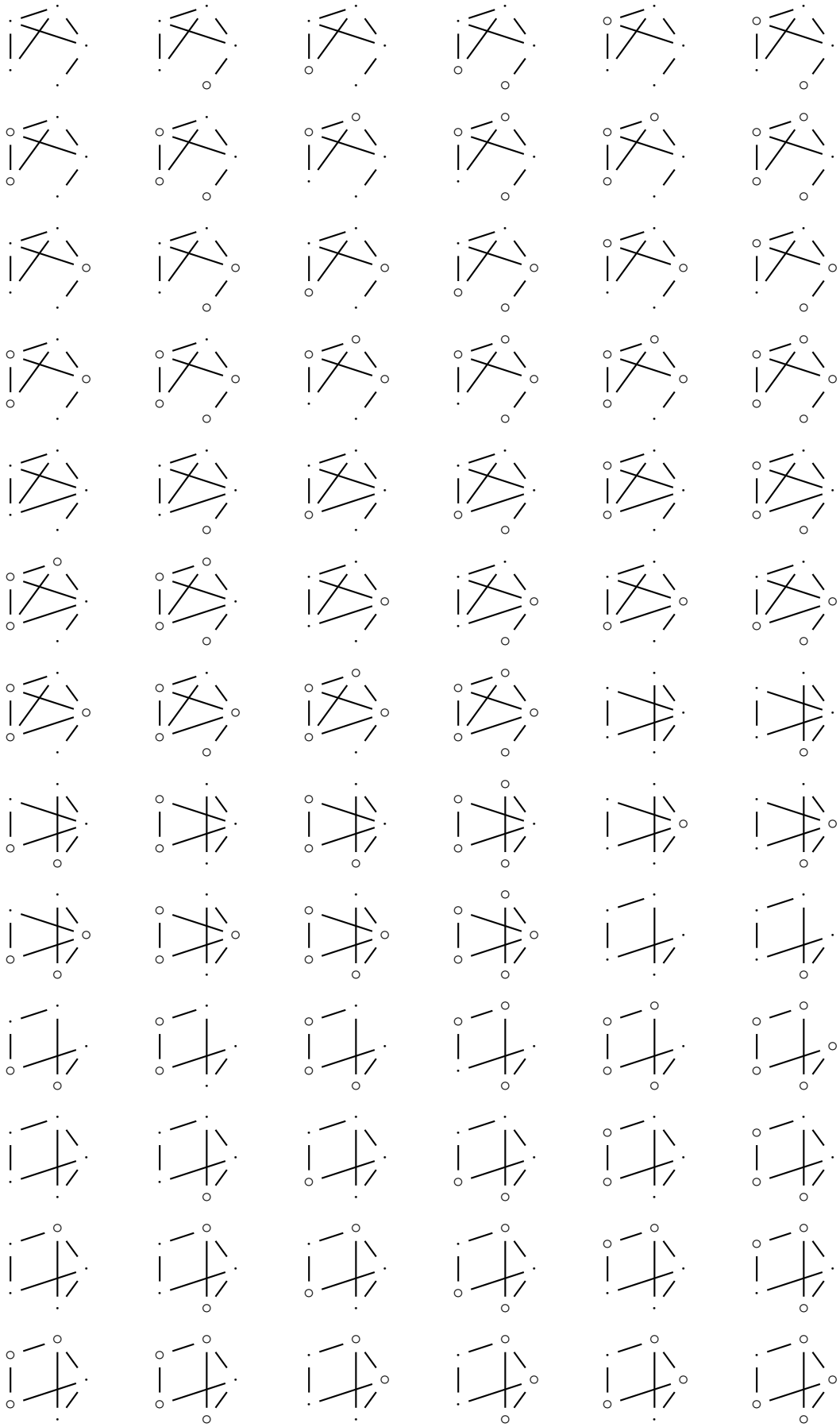


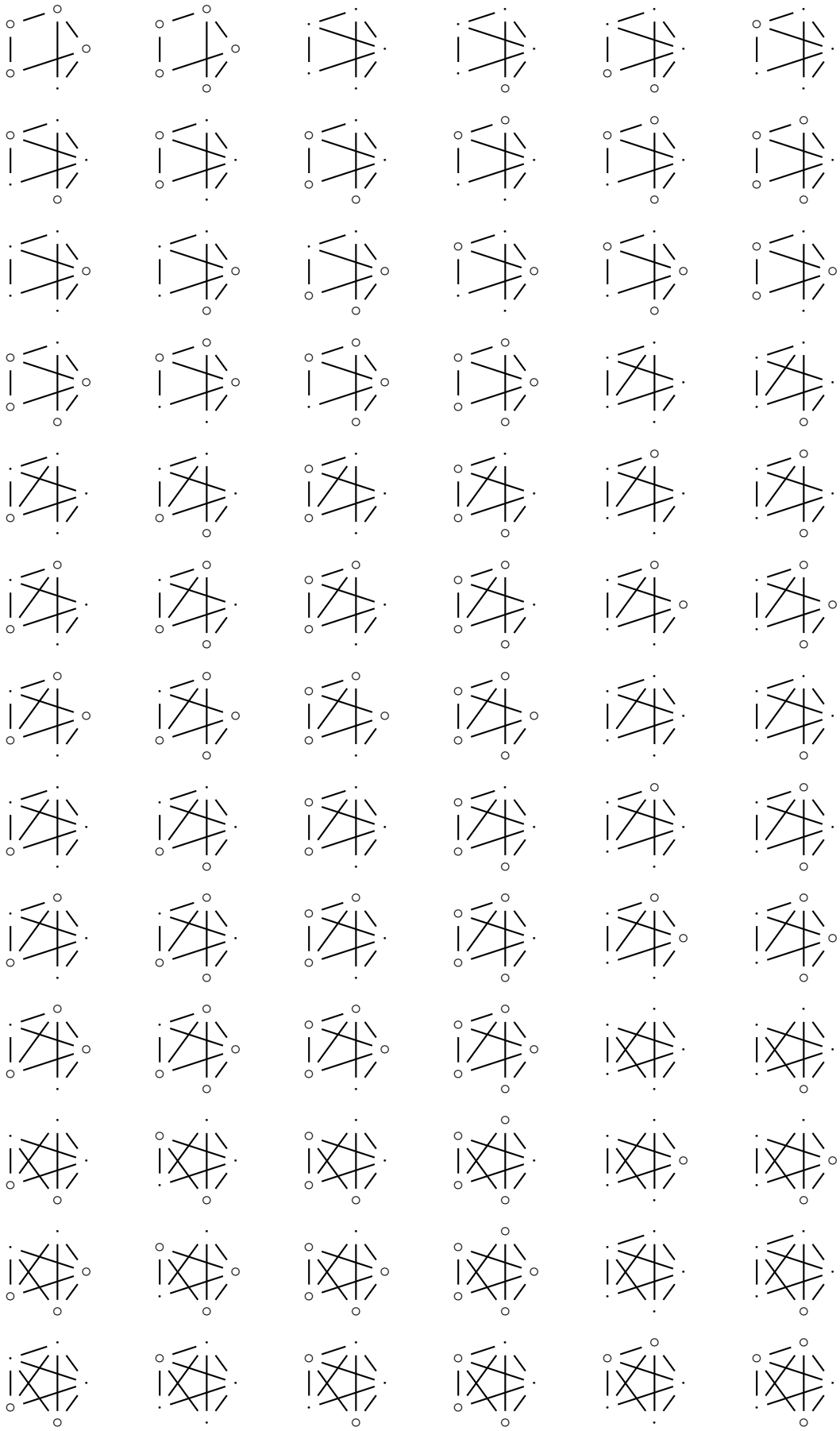


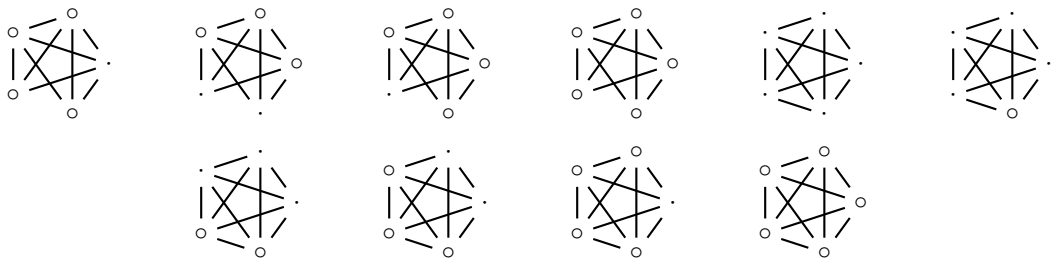












References

[1] S. Axler. *Linear Algebra Done Right*. Springer Cham Heidelberg New York Dordrecht London, third edition, 2015.

[2] L. Bedratyuk. A new formula for the generating function of the numbers of simple graphs. *Comptes rendus de l'Académie bulgare des Sciences*, 69, 12 2015.

[3] J. H. Silverman. *Abstract Algebra An Integrated Approach*, volume 55 of *Pure and applied undergraduate texts, 1943-9334*.

[4] N. J. A. Sloane. Number of graphs on n unlabeled nodes.

[5] D. B. West. *Introduction to Graph Theory*. Pearson Education, second edition, 2001.